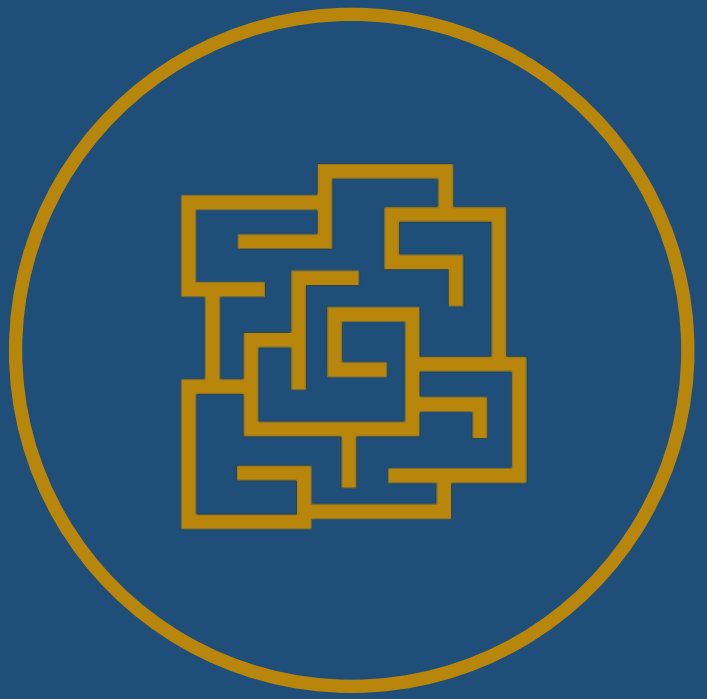
INFORMATIK LK · WELT 3 · DS 18–23

Das Labyrinth

Rekursion, 2D-Karten, Teile und herrsche



L3.1 ····· L3.2 ····· Rennen ····· L3.3 ····· BOSS

Das Labyrinth im Überblick



DS 18

IDE-Level 3.1
Rekursion

Basisfall, Aufrufstapel, Fibonacci, Fraktale

Lab 3.1 · 3.2



DS 19

IDE-Level 3.2
Dungeon-Karte

2D-Reihung, Nachbarn, Nebel lüften

Lab 3.3



DS 20 · Kür

IDE 3.2 Experte
Backtracking

Wegsuche und Dungeon-Rennen

Lab 3.4 · Rennen



DS 21

IDE-Level 3.3
Teile und herrsche

binäre Suche, Maximum, Mergesort

Lab 3.5



DS 22

IDE 3.3 Experte
Quicksort

zerlegen, fremde Verfahren analysieren

Lab 3.6



DS 23

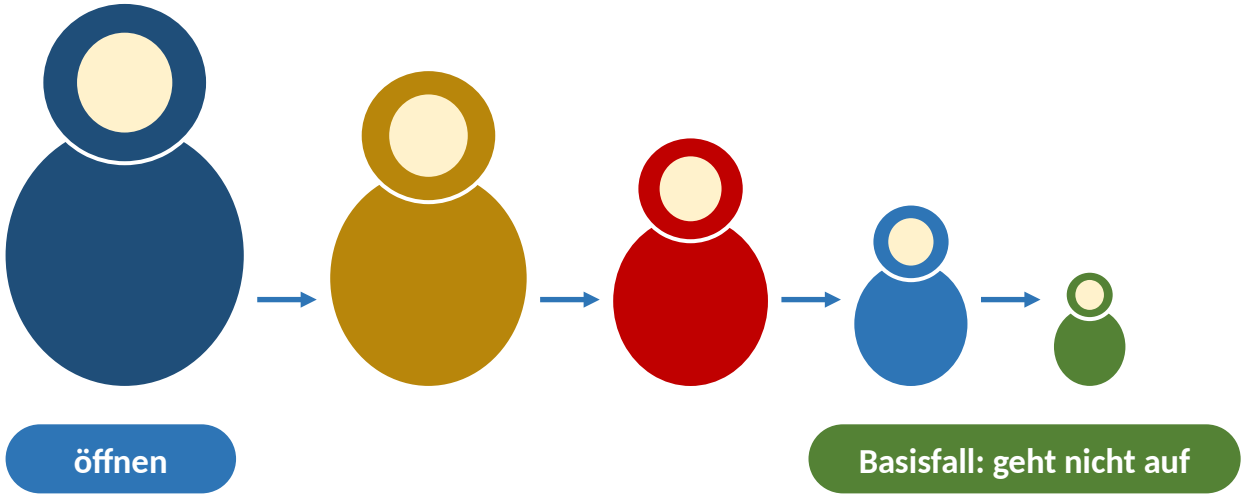
Boss 3
Kristallhöhle

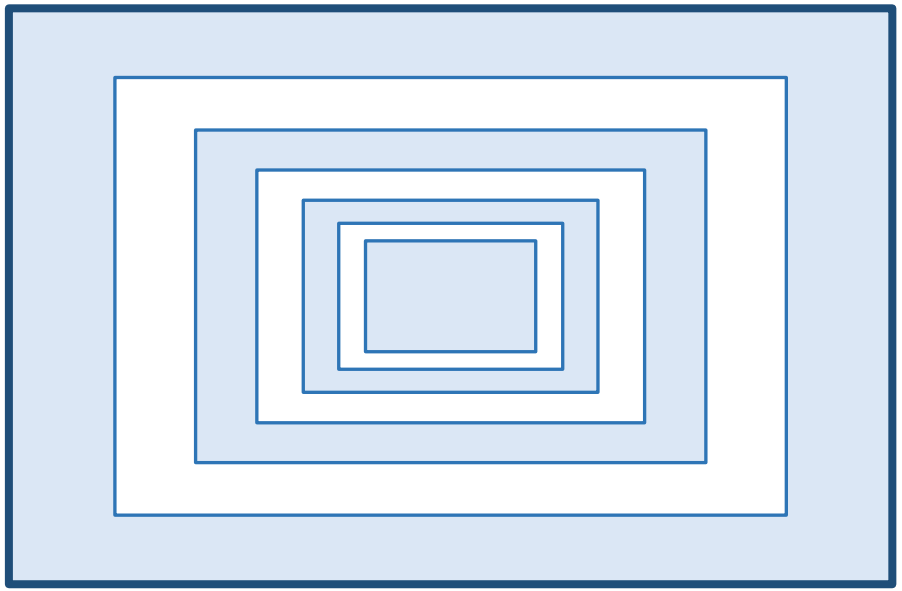
Abi auf Zeit: 24 BE in 60 min

+ Online-Boss

Roter Faden: Ein großes Problem wird auf ein kleineres derselben Art zurückgeführt – bis es ganz einfach ist.

Rekursion unplugged: Puppe in der Puppe





Zwei Spiegel gegenüber: Bild im Bild im Bild …

```
anzahl(puppe):
  falls puppe nicht zu öffnen: gib 1 zurück
  sonst: gib 1 + anzahl(innere Puppe) zurück
```

Achtung: Der Spiegel hört nie auf – eine Rekursion muss enden.

Rekursion: Eine Methode löst ein Problem, indem sie sich selbst für ein kleineres Problem derselben Art aufruft.

Basisfall und Rekursionsschritt

$0! = 1$ Basisfall

$n! = n \cdot (n - 1)!$ Rekursionsschritt ($n \geq 1$)

$4! = 4 \cdot 3! = 4 \cdot 3 \cdot 2! = 4 \cdot 3 \cdot 2 \cdot 1! = 4 \cdot 3 \cdot 2 \cdot 1 \cdot 0! = 24$

```
public static int fakultaet(int n) {
    if (n == 0) {
        return 1;                    // Basisfall
    }
    return n * fakultaet(n - 1);  // Rekursionsschritt
}
```

Denkanstoß: Nicht in die Tiefe denken – vertraue darauf, dass fakultaet(n − 1) schon richtig rechnet.

1 **Basisfall zuerst**
Wann steht die Antwort sofort fest? Dann „gib … zurück“ – ohne Selbstaufruf.

2 **Problem wird kleiner**
Der Selbstaufruf bekommt ein echt kleineres Problem: n − 1, i + 1, halbe Reihung.

3 **Ergebnis zusammensetzen**
Was macht dieser Aufruf aus dem Teilergebnis? Hier: n · fakultaet(n − 1).

Der Aufrufstapel: fakultaet(4)

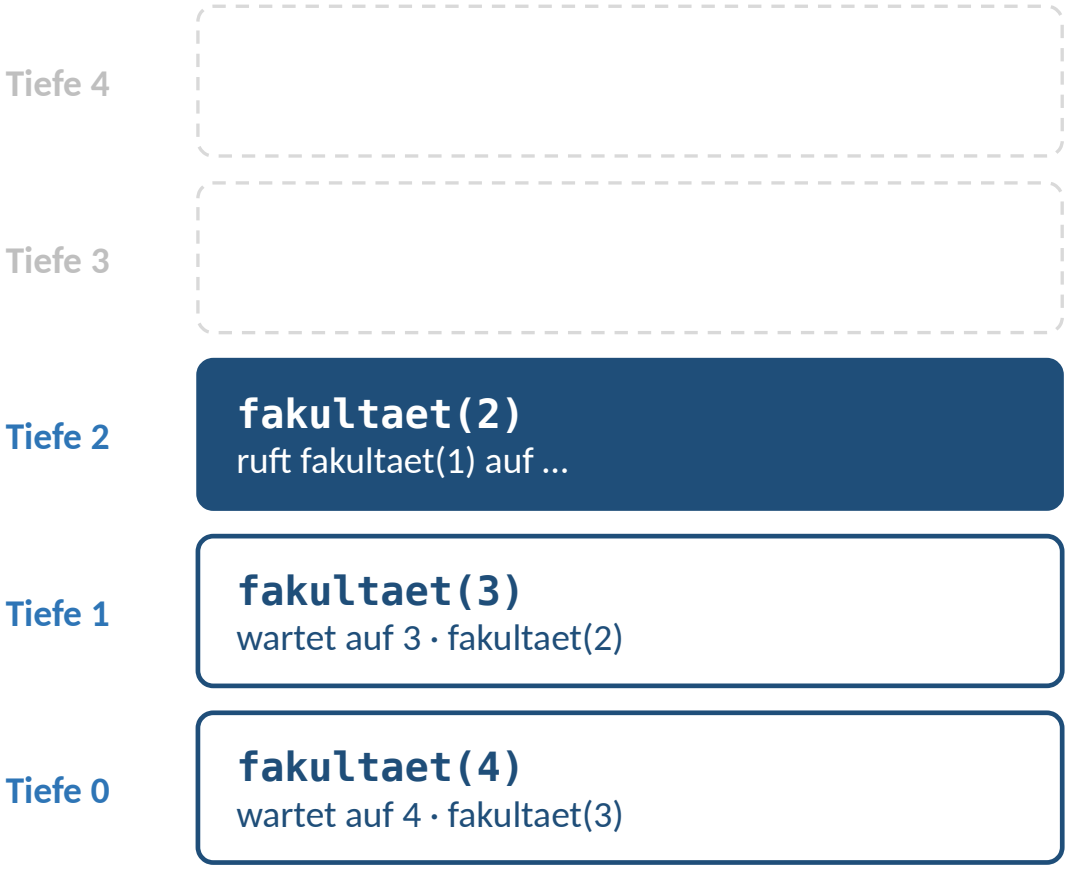
```java
public static int fakultaet(int n) {
    if (n == 0) {
        return 1;                    // Basisfall
    }
    return n * fakultaet(n - 1);  // Rekursionsschritt
}
```

1 Abstieg — 2 Basisfall — 3 Aufstieg — 4 Ergebnis

Abstieg: Jeder Aufruf legt oben einen neuen Rahmen mit eigenem n auf den Stapel. fakultaet(4) kann noch nicht rechnen – es wartet auf fakultaet(3).

gold umrandet: die Zeile, die gerade ausgeführt wird

Aufrufstapel (oben: aktiver Aufruf)



Der Aufrufstapel: fakultaet(4)

```java
public static int fakultaet(int n) {
    if (n == 0) {
        return 1;                    // Basisfall
    }
    return n * fakultaet(n - 1);  // Rekursionsschritt
}
```

1 Abstieg — **2 Basisfall** — 3 Aufstieg — 4 Ergebnis

Basisfall: fakultaet(0) ruft niemanden mehr auf und liefert sofort 1. Hier ist der Stapel am höchsten: Rekursionstiefe 4.

gold umrandet: die Zeile, die gerade ausgeführt wird

Aufrufstapel (oben: aktiver Aufruf)

Tiefe 4	**`fakultaet(0)`** n = 0 → gib 1 zurück
Tiefe 3	**`fakultaet(1)`** wartet auf 1 · fakultaet(0)
Tiefe 2	**`fakultaet(2)`** wartet auf 2 · fakultaet(1)
Tiefe 1	**`fakultaet(3)`** wartet auf 3 · fakultaet(2)
Tiefe 0	**`fakultaet(4)`** wartet auf 4 · fakultaet(3)

Der Aufrufstapel: fakultaet(4)

```java
public static int fakultaet(int n) {
    if (n == 0) {
        return 1;                    // Basisfall
    }
    return n * fakultaet(n - 1);  // Rekursionsschritt
}
```

1 Abstieg — 2 Basisfall — **3 Aufstieg** — 4 Ergebnis

Aufstieg: Der oberste Rahmen verschwindet, sein Ergebnis wird unten eingesetzt: fakultaet(1) = 1 · 1 = 1, dann fakultaet(2) = 2 · 1 = 2.

gold umrandet: die Zeile, die gerade ausgeführt wird

Aufrufstapel (oben: aktiver Aufruf)



Der Aufrufstapel: fakultaet(4)

```java
public static int fakultaet(int n) {
    if (n == 0) {
        return 1;                    // Basisfall
    }
    return n * fakultaet(n - 1);  // Rekursionsschritt
}
```

1 Abstieg — 2 Basisfall — 3 Aufstieg — **4 Ergebnis**

Ergebnis: fakultaet(3) = 3 · 2 = 6, fakultaet(4) = 4 · 6 = 24.
Insgesamt 5 Aufrufe, größte Rekursionstiefe 4.

fakultaet(4) = 24 · 5 Aufrufe · größte Tiefe 4

Aufrufstapel (oben: aktiver Aufruf)



Die Rekursionstabelle mit Rekursionstiefe

Rekursionstiefe	Aufruf	n	Rückgabe
0	`fakultaet(4)`	4	**4 · 6 = 24**
1	`fakultaet(3)`	3	**3 · 2 = 6**
2	`fakultaet(2)`	2	**2 · 1 = 2**
3	`fakultaet(1)`	1	**1 · 1 = 1**
4	`fakultaet(0)`	0	**1 (Basisfall)**

1. Abstieg: Aufrufe von oben nach unten eintragen – bis zum Basisfall.
2. Aufstieg: Rückgaben von unten nach oben ausrechnen.

Aufrufe zählen: 5 Aufrufe, größte Tiefe 4 – allgemein: n + 1 Aufrufe, Tiefe n

Achtung Abi:
Der Erstaufruf hat die Rekursionstiefe 0. Jeder Selbstaufruf erhöht die Tiefe um 1.

Jetzt du (Papier)
Rekursionstabelle für dual(13): Tiefe, Aufruf, n, n / 2, n % 2, Rückgabe.

Rekursive Struktogramme nach EH §5

fakultaet(n: Ganzzahl): Ganzzahl	
n = 0	
wahr	falsch
gib 1 zurück	gib n · fakultaet(n − 1) zurück
// Basisfall	*// Rekursionsschritt*

Selbstaufruf im „gib … zurück“

maximumAb(r: Reihung von Ganzzahl, i: Ganzzahl): Ganzzahl		
i = Länge von r − 1		
wahr	falsch	
gib r[i] zurück	maxRest ← maximumAb(r, i + 1)	
// Basisfall	r[i] > maxRest	
	wahr	falsch
	gib r[i] zurück	gib maxRest zurück

Selbstaufruf als Zuweisung – maximumAb({3, 8, 2, 5}, 0) liefert 8

Merke: Kein Sondersymbol – der Selbstaufruf ist eine ganz normale Anweisung.

Achtung Abi: Kopf mit Signatur · Basisfall als Verzweigung zuerst · jeder Pfad endet mit „gib … zurück“ · Teilproblem echt kleiner.

Rekursion oder Schleife?

rekursiv

```
public static int summeBis(int n) {
    if (n == 0) {
        return 0;              // Basisfall
    }
    return n + summeBis(n - 1);
}
```

summeBis(10) = 55

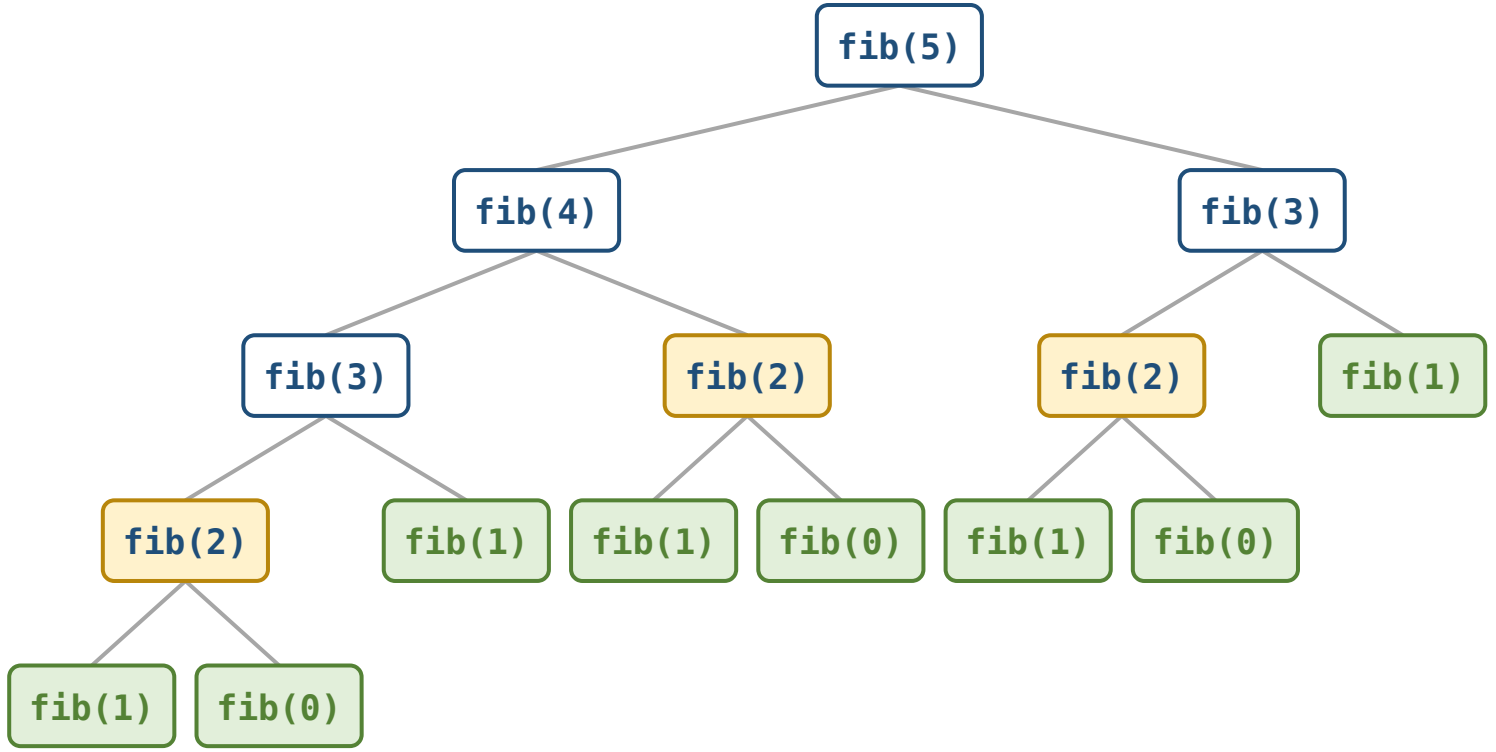
iterativ (mit Schleife)

```
public static int summeBisIterativ(int n) {
    int summe = 0;
    for (int i = 1; i <= n; i++) {
        summe = summe + i;
    }
    return summe;
}
```

summeBisIterativ(10) = 55

	Rekursion	Schleife
Wiederholung	Selbstaufruf	Schleifenrumpf
Abbruch	Basisfall	Schleifenbedingung
Zwischenstand	je Aufruf ein Rahmen auf dem Stapel	eine Variable (summe)
Fehlerbild	**StackOverflowError**	**Endlosschleife**
stark bei	Labyrinth, Bäume, Teile und herrsche	einfachem Zählen und Summieren

Baumrekursion: der Aufrufbaum von fibonacci(5)



grün: Basisfall · gold: fib(2) wird 3-mal neu berechnet

n	0	1	2	3	4	5	6	7	8	9	10
Aufrufe	1	1	3	5	9	**15**	25	41	67	109	177

```java
public static int fibonacci(int n) {
    aufrufe++;
    if (n < 2) {
        return n;
    }
    return fibonacci(n - 1)
         + fibonacci(n - 2);
}
```

Aufrufe bestimmen

A(0) = A(1) = 1

A(n) = A(n − 1) + A(n − 2) + 1

A(5) = 9 + 5 + 1 = 15

Verschwendung: n + 1 → etwa 1,6-mal so viele Aufrufe. Größte Tiefe: n − 1 = 4.

Wenn der Stapel überläuft: StackOverflowError

1 Basisfall fehlt

```
public static int fakultaet(int n) {
    return n * fakultaet(n - 1);
}
```

2 Problem wird nicht kleiner

```
    return n * fakultaet(n);
```

3 Basisfall wird übersprungen

```
fakultaet(-1)   // n: -1, -2, -3, … nie 0
```

```
Exception in thread "main" java.lang.StackOverflowError
```

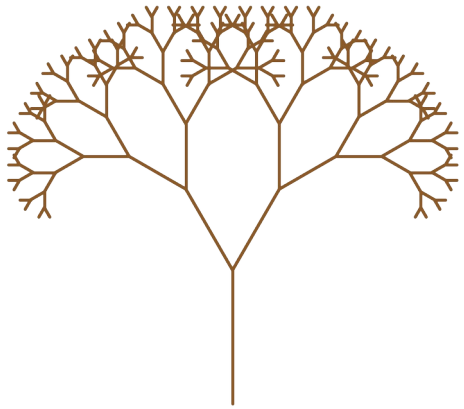
alle drei Fälle mit javac ausgeführt

Merke: Jeder Aufruf belegt Platz auf dem Stapel.

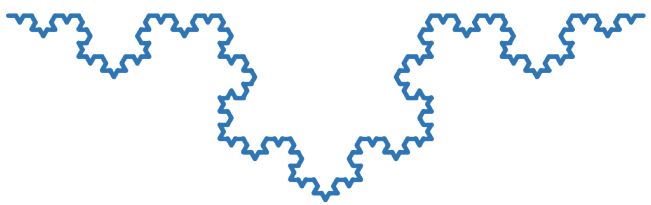
Basisfall n = 0 wurde verpasst – der Stapel wächst, bis er voll ist.



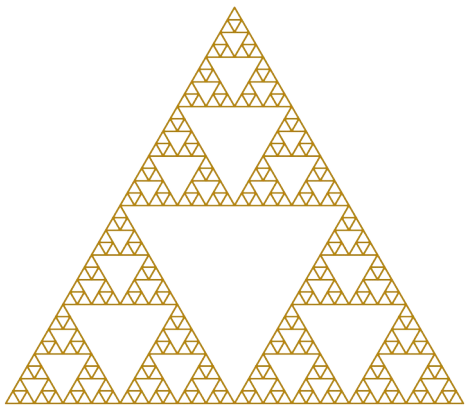
Deko für den Dungeon: rekursive Fraktale



Fraktal-Baum
Tiefe 8 · 2 Selbstaufrufe je Aufruf



Koch-Kurve („Eiszapfen“)
Tiefe 4 · 4 Selbstaufrufe je Aufruf

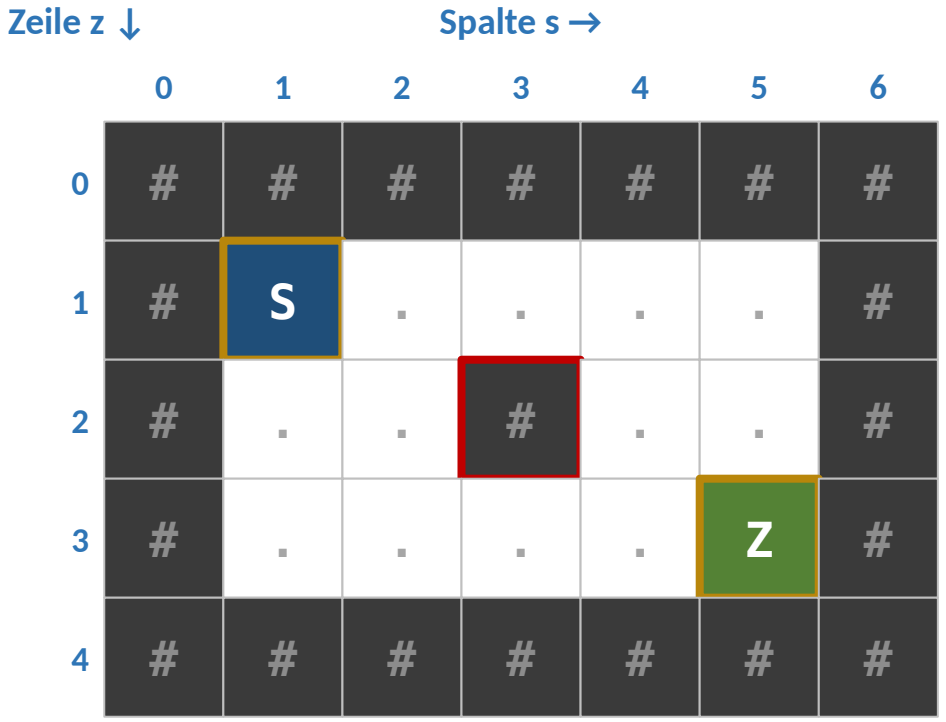


Sierpinski-Dreieck
Tiefe 5 · 3 Selbstaufrufe je Aufruf

Aufrufe	k	Tiefe 0	Tiefe 1	Tiefe 2	Tiefe 3	Tiefe 4	Tiefe 5	Formel A(n)
Baum	2	1	3	7	15	31	63	`2^(n+1) – 1`
Koch-Kurve	4	1	5	21	85	341	1365	`(4^(n+1) – 1) / 3`
Sierpinski	3	1	4	13	40	121	364	`(3^(n+1) – 1) / 2`

A(0) = 1, A(n) = 1 + k · A(n − 1): Tiefe + 1 → etwa k-mal so viele Aufrufe (exponentiell).

Die Dungeon-Karte als 2D-Reihung



`feld[1][1] = 'S'` `feld[3][5] = 'Z'` `feld[2][3] = '#'`

Testkarte „Höhle“: 5 Zeilen, 7 Spalten

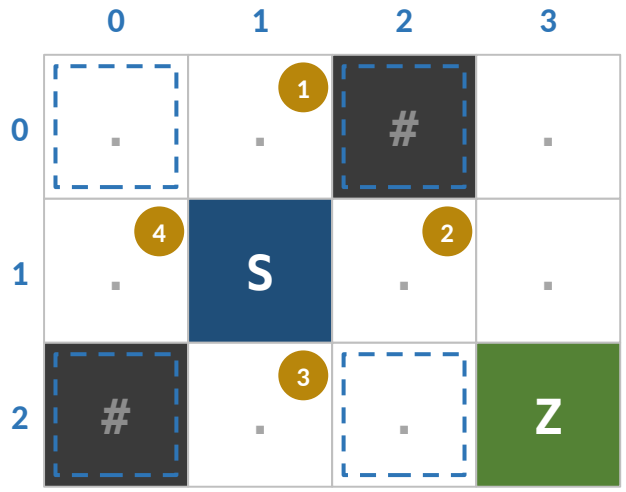
```
private char[][] feld;       // feld[z][s]
feld.length                  // 5 Zeilen
feld[0].length               // 7 Spalten
```

```
public int anzahlWaende() {
    int anzahl = 0;
    for (int z = 0; z < feld.length; z++) {
        for (int s = 0; s < feld[z].length; s++) {
            if (feld[z][s] == '#') {
                anzahl++;
            }
        }
    }
    return anzahl;
}
```

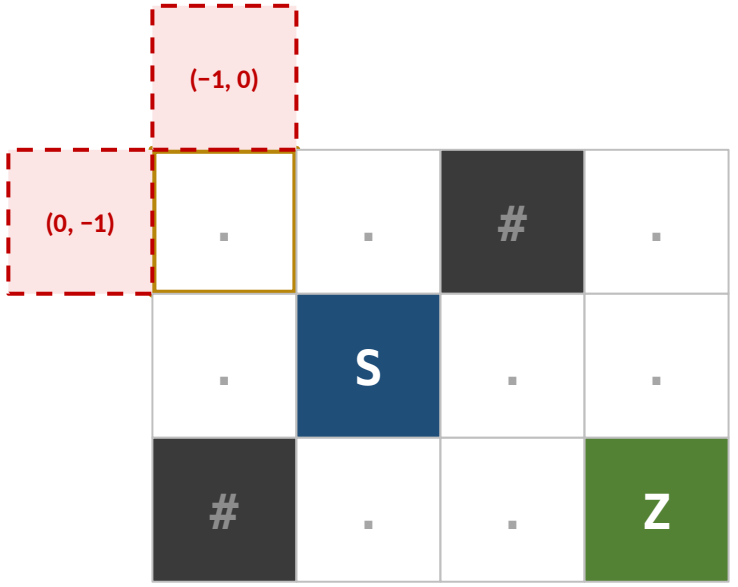
Höhle: anzahlWaende() = 21

Achtung Abi: erst Zeile, dann Spalte: feld[z][s] – nicht x, y! Zeichen vergleicht man mit ==.

Nachbarn und Rand: die Lichtung ohne Randmauer



4er: 1 oben, 2 rechts, 3 unten, 4 links
8er: dazu die Diagonalen (gestrichelt)



Ecke (0, 0): oben und links liegen außerhalb

`anzahlBegehbareNachbarn`

Feld	Nachbarn
(1, 1)	4
(0, 0)	2
(2, 3)	2
(0, 3)	1
(1, 2)	3 (8er: 7)

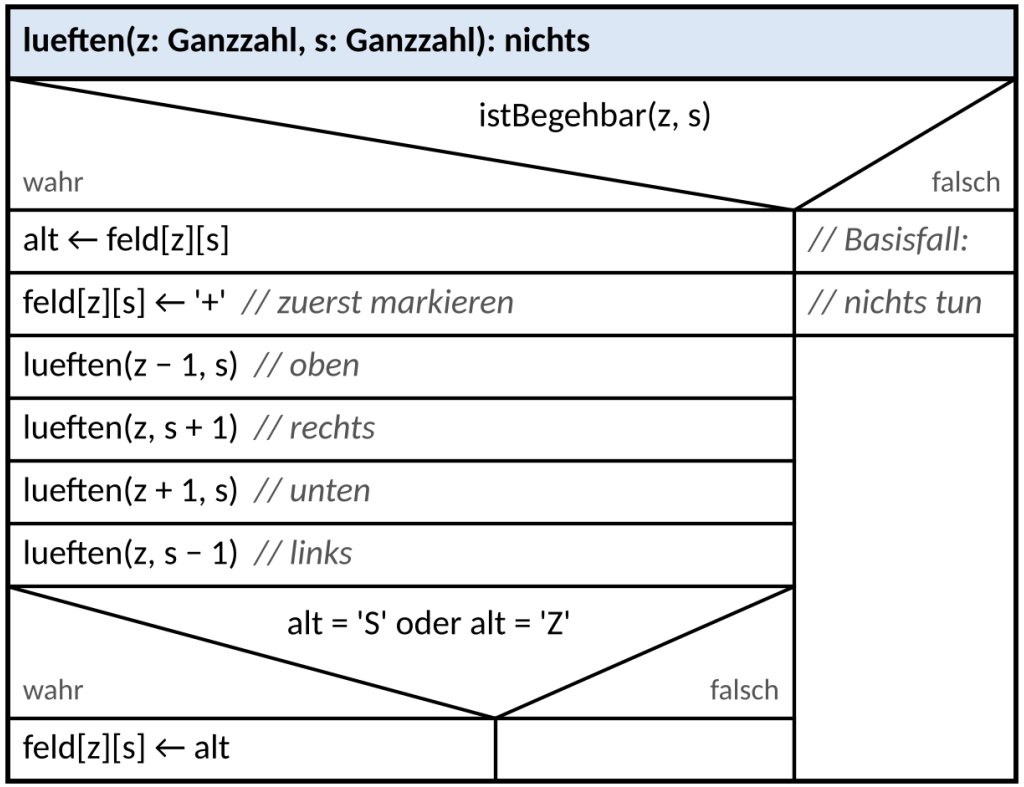
```
public boolean istBegehbar(int z, int s) {
    if (!istImFeld(z, s)) {
        return false;     // zuerst den Rand pruefen!
    }
    char c = feld[z][s];
    return c == '.' || c == 'S' || c == 'Z';
}
```

Achtung: feld[−1][0] gibt es nicht → ArrayIndexOutOfBoundsException. Erst istImFeld, dann zugreifen.

Nebel lüften: rekursiver Flood-Fill

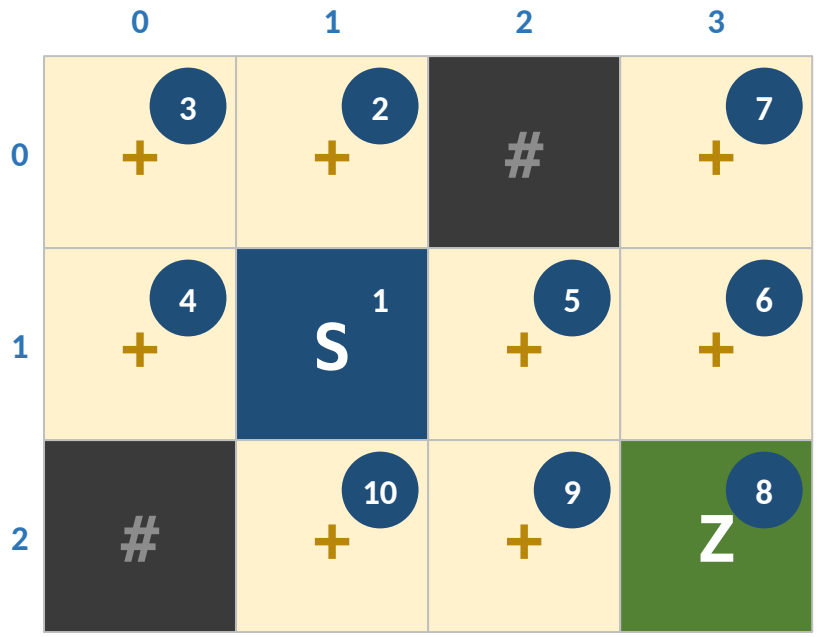
```
public void lueften(int z, int s) {
    if (!istBegehbar(z, s)) {
        return;  // Basisfall: Rand, Wand oder schon besucht
    }
    char alt = feld[z][s];
    feld[z][s] = '+';       // ZUERST als besucht markieren
    lueften(z - 1, s);      // oben
    lueften(z, s + 1);      // rechts
    lueften(z + 1, s);      // unten
    lueften(z, s - 1);      // links
    if (alt == 'S' || alt == 'Z') {
        feld[z][s] = alt;  // S und Z wieder sichtbar machen
    }
}
```

Besucht-Markierung: erst markieren, dann die Nachbarn lüften. Sonst ruft (1, 1) den Nachbarn (1, 2) auf, der wieder (1, 1) … → StackOverflowError.



S und Z gelten kurz als '+' und werden am Ende zurückgesetzt.

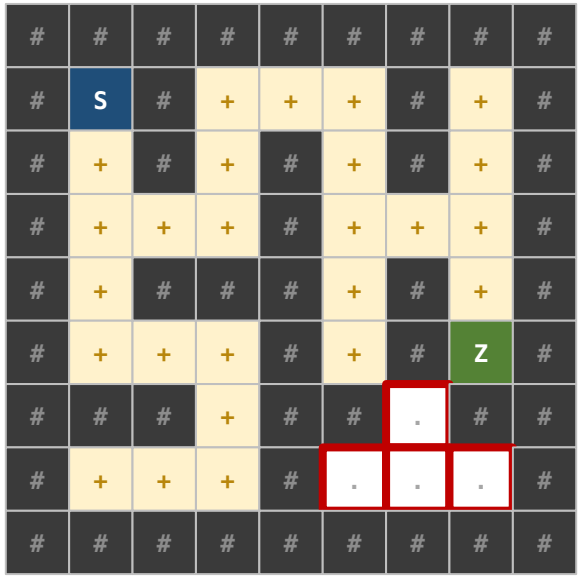
Flood-Fill Schritt für Schritt: lueften(1, 1)



Lichtung nach lueften(1, 1) · Nummer = Reihenfolge der Markierung

10 markiert · 41 Aufrufe = 4 · 10 + 1

Nr	Feld	Weg
1	(1, 1)	Start S
2	(0, 1)	von 1 oben
3	(0, 0)	von 2 links
4	(1, 0)	von 3 unten
5	(1, 2)	von 1 rechts
6	(1, 3)	von 5 rechts
7	(0, 3)	von 6 oben
8	(2, 3)	von 6 unten: Z
9	(2, 2)	von 8 links
10	(2, 1)	von 9 links



Labyrinth: 25 gelüftet – die Kammer unten rechts hängt nur über Eck am Gang und bleibt im Nebel.

Merke: Jedes markierte Feld ruft vier Nachbarn auf, dazu der Erstaufruf. Die Reihenfolge ändert nur die Nummern, nicht das Ergebnis.

Backtracking: Weg suchen, Sackgasse zurücknehmen

```java
public boolean findeWeg(int z, int s) {
    schritte++;
    if (!istImFeld(z, s)) {
        return false;      // Basisfall: ausserhalb
    }
    char alt = feld[z][s];
    if (alt == 'Z') {
        return true;       // Basisfall: Ziel gefunden
    }
    if (alt != '.' && alt != 'S') {
        return false;      // Basisfall: Wand oder schon besucht
    }
    feld[z][s] = '*';      // vorlaeufig auf den Weg setzen      1
    if (findeWeg(z - 1, s) || findeWeg(z, s + 1)               2
            || findeWeg(z + 1, s) || findeWeg(z, s - 1)) {
        if (alt == 'S') {
            feld[z][s] = 'S';  // Start bleibt sichtbar
        }
        return true;
    }
    feld[z][s] = 'x';      // Schritt zuruecknehmen: Sackgasse   3
    if (alt == 'S') {
        feld[z][s] = 'S';
    }
    return false;
}
```

#	#	#	#	#	#	#	#	#
#	S	#	*	*	*	#	x	#
#	*	#	*	#	*	#	x	#
#	*	*	*	#	*	*	*	#
#	.	#	#	#	.	#	*	#
#	.	.	.	#	.	#	Z	#
#	#	#	.	#	#	.	#	#
#	.	.	.	#	.	.	.	#
#	#	#	#	#	#	#	#	#

- `*` Weg: 13 Felder
- `x` Sackgasse
- `.` nie besucht
- `S` Start
- `Z` Ziel

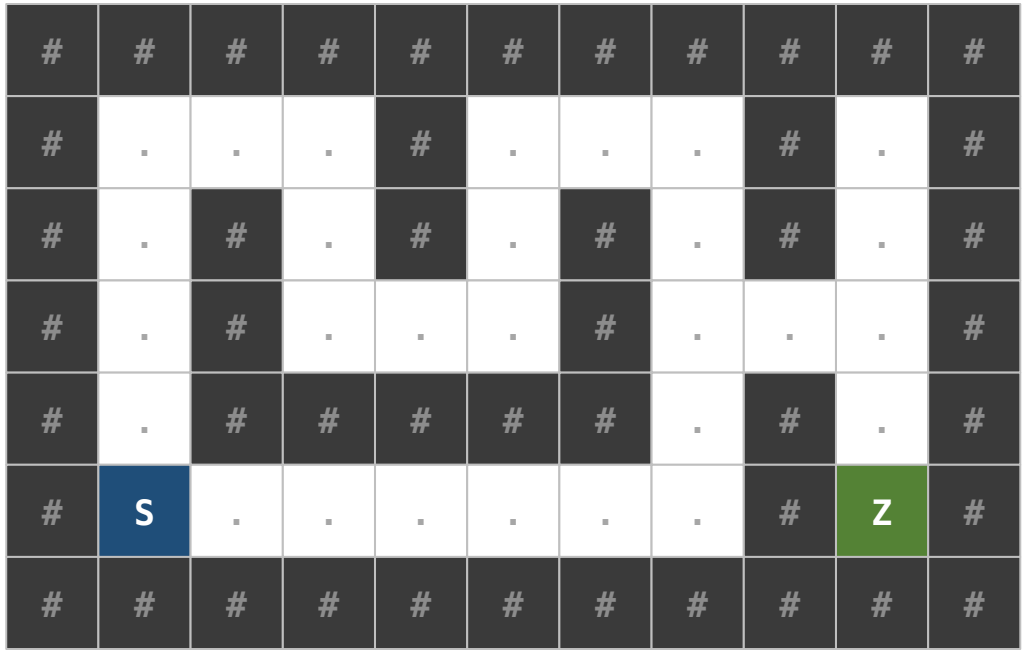
`findeWeg(1, 1)`
= true · 41 Aufrufe

1 vorläufig markieren
Feld wird '*'

2 Nachbarn versuchen
oben, rechts, unten, links – beim ersten true fertig

3 zurücknehmen
kein Nachbar führt zum Ziel: 'x', false

Das Dungeon-Rennen



Rennkarte 1 (klein, 7 × 11) · dazu Karte 2 (11 × 15), Karte 3 (15 × 21) und drei unbekannte Kontrollkarten

```
RENNEN-ERGEBNIS: Schritte = …, gueltig = ja
```

letzte Zeile der Konsole nach Rennen.starten()

betrete(z, s) = 1 Schritt
auch S und Z zählen · nur Nachbarfelder, kein Feld zweimal

verlasse(z, s) ist kostenlos
Sackgasse markieren und ein Feld zurück

Nebel
istBegehbar, istZiel nur für Standort und vier Nachbarn

Kompass erlaubt
zielZeile(), zielSpalte() zeigen immer zum Ziel

Wertung
Summe der Schritte auf 3 Rennkarten – weniger ist besser

Ungültig: feste Wege für bekannte Karten, Math.random, Rennen verändern, Kontrollkarte nicht gelöst.

Vom Konsolen-Ergebnis an die Tafel

Team	Strategie	Schritte	gültig?
Team 1			
Team 2			
Team 3			
Team 4			
Team 5			

Ergebnistabelle an der Tafel: gültige Ergebnisse nach Schritten aufsteigend, ungültige markiert am Ende

Wertung: Summe der Schritte auf den Rennkarten 1–3 · weniger ist besser

1. **Rennen.starten()**
   3 Rennkarten + 3 Kontrollkarten, am Ende die Ergebniszeile
2. **Ein Ergebnis je Team**
   das Team entscheidet, welche Version von suche antritt
3. **Eintrag an der Tafel**
   Schritte und gültig ja/nein – kurzer Blick in den Code
4. **Strategie vorstellen**
   Warum weniger Schritte – und auf welcher Karte Pech?

Team-Wettbewerb im Unterricht
ohne Lab-Wertung – es zählt die beste Strategie

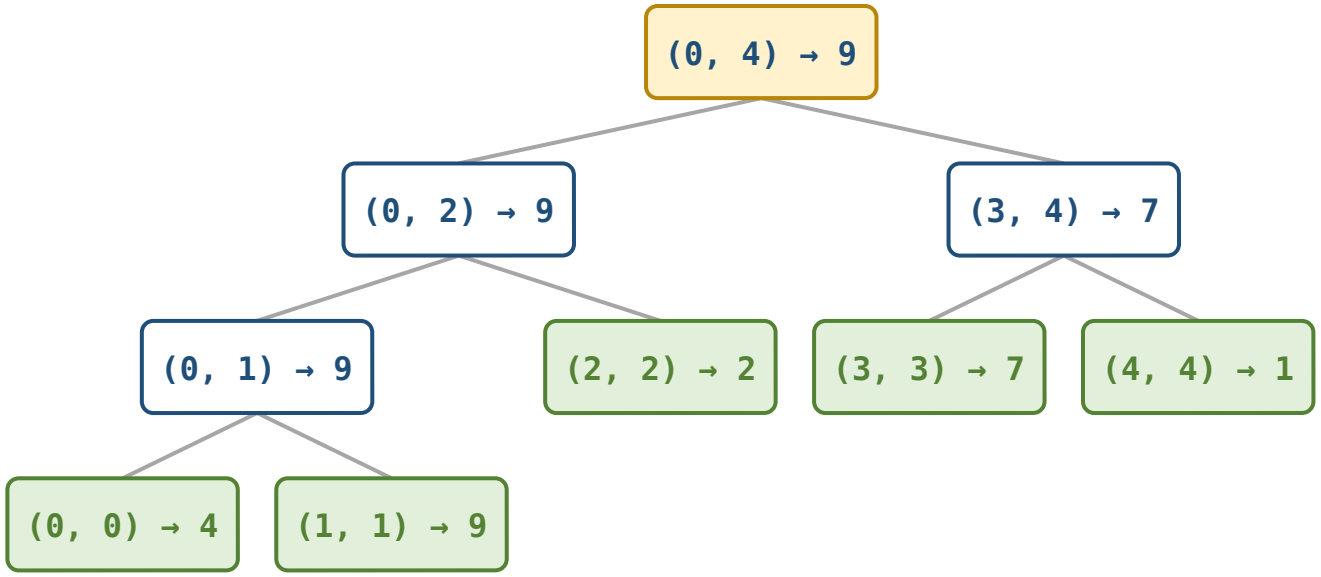
Teile und herrsche

TEILEN
Problem in kleinere Teilprobleme derselben Art zerlegen – meist halbieren

HERRSCHEN
Teilprobleme rekursiv lösen – ganz kleine direkt (Basisfall)

VERBINDEN
Teillösungen zum Gesamtergebnis zusammensetzen

`maximumTH(r, 0, 4)` mit r = {4, 9, 2, 7, 1}



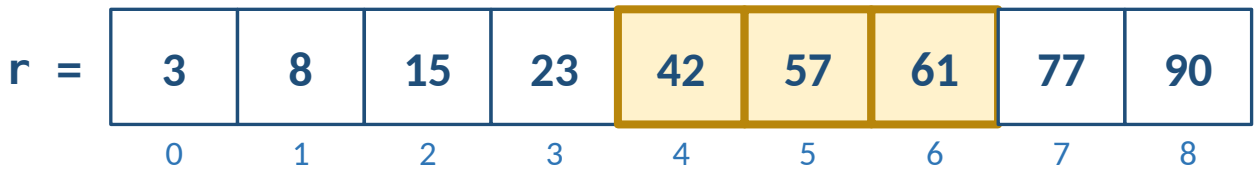
Teilen in der Mitte: mitte ← (von + bis) / 2; links von … mitte, rechts mitte + 1 … bis.

9 Aufrufe = 2 · 5 − 1 – jedes Element einmal Basisfall

Achtung: rechts ab mitte statt mitte + 1 → StackOverflowError

Binäre Suche – rekursiv

```java
public static int binaerRek(int[] r, int x, int lo, int hi) {
    if (lo > hi) {
        return -1;                    // Basisfall: Bereich leer
    }
    int mid = (lo + hi) / 2;
    if (r[mid] == x) {
        return mid;                   // Basisfall: gefunden
    }
    if (r[mid] < x) {
        return binaerRek(r, x, mid + 1, hi);  // rechts weitersuchen
    }
    return binaerRek(r, x, lo, mid - 1);      // links weitersuchen
}
```



gold: die geprüften Mitten (Tiefe 0: 42, Tiefe 1: 61, Tiefe 2: 57)

`binaerRek(r, 50, 0, 8)`

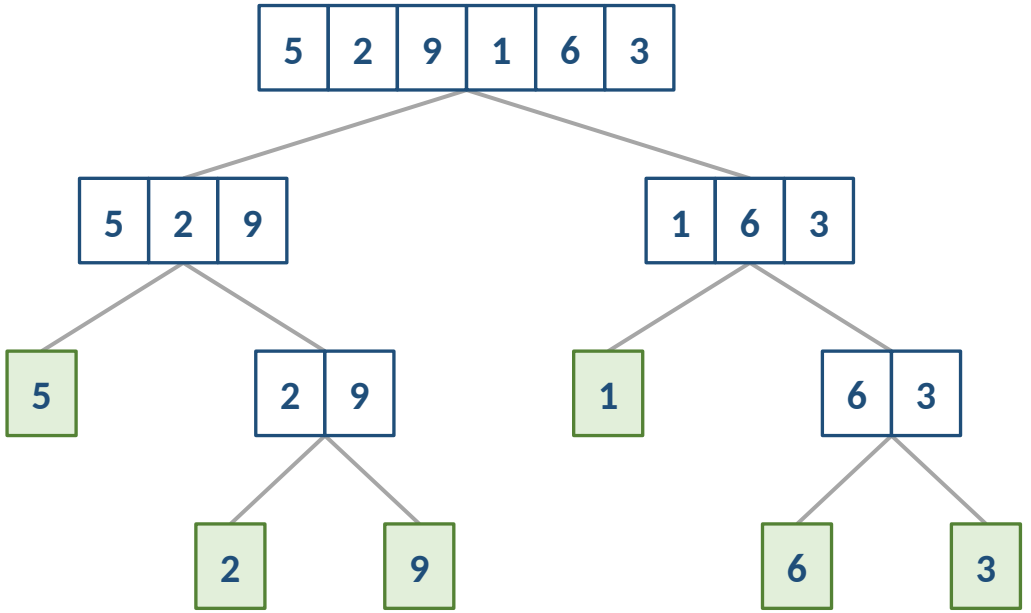
Tiefe	lo	hi	mid	r[mid]	Entscheidung
0	0	8	4	42	42 < 50 → rechts
1	5	8	6	61	61 > 50 → links
2	5	5	5	57	57 > 50 → links
3	5	4	–	–	lo > hi → −1

Start: **`binaer(r, x)`** ruft binaerRek(r, x, 0, r.length − 1) auf.

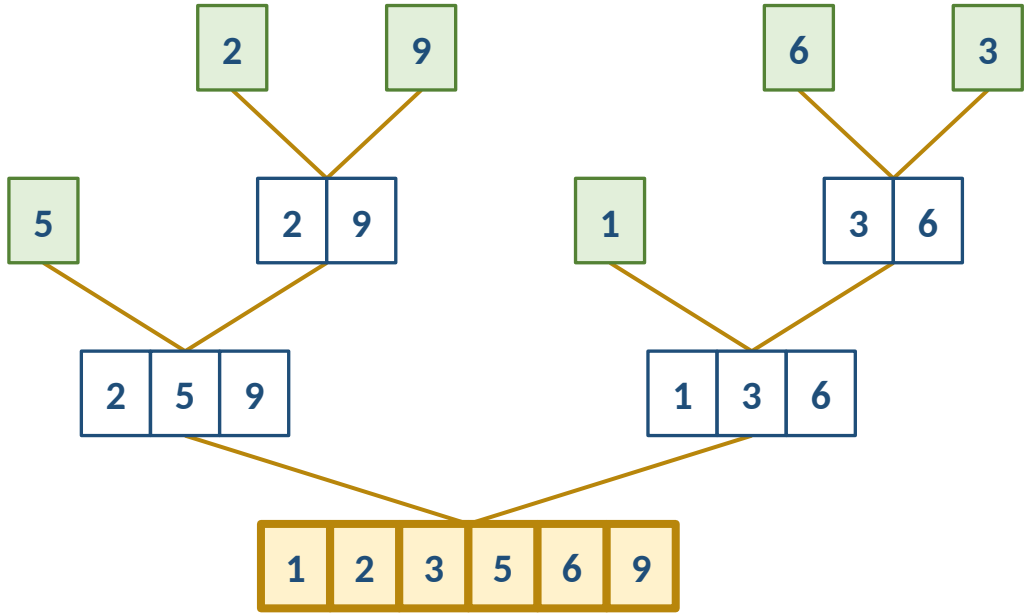
Achtung: mid + 1 und mid − 1 – sonst wird der Bereich nicht kleiner.

Mergesort: Teilungsbaum und Verschmelzungsbaum

TEILEN: mitte = Länge / 2



VERBINDEN: verschmelze(links, rechts)

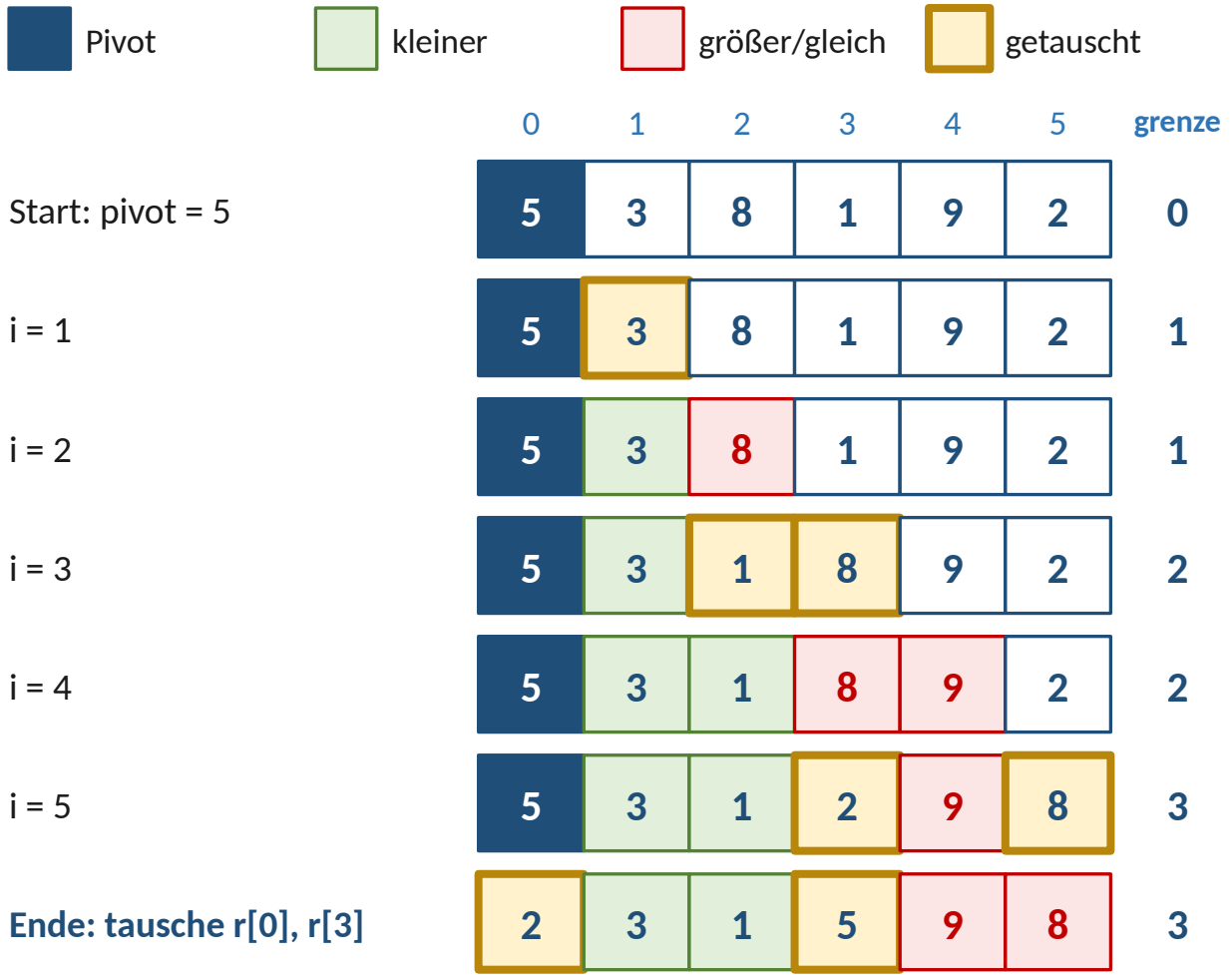


mergeSort({5, 2, 9, 1, 6, 3}): 11 Aufrufe, 10 Vergleiche

Achtung: mitte = (Länge − 1) / 2 → bei Länge 2 bleibt links leer: StackOverflowError

Quicksort: zerlege Schritt für Schritt

```java
public static int zerlege(int[] r, int lo, int hi) {
    int pivot = r[lo];
    int grenze = lo; // → Tabellenzeile (Startwerte)
    for (int i = lo + 1; i <= hi; i++) {
        vergleiche++;
        if (r[i] < pivot) {
            grenze++;
            int h = r[grenze];
            r[grenze] = r[i];
            r[i] = h;
        } // → Tabellenzeile
    }
    int hilf = r[lo];
    r[lo] = r[grenze];
    r[grenze] = hilf; // → Tabellenzeile
    return grenze;
}
```



Ergebnis: Pivot 5 an Index 3 – links nur Kleinere, rechts nur Größere. 5 Vergleiche.

Quicksort rekursiv – und der Vergleich der Verfahren

```
public static void quickSort(int[] r, int lo, int hi) {
    if (lo < hi) {
        int p = zerlege(r, lo, hi);
        quickSort(r, lo, p - 1);
        quickSort(r, p + 1, hi);
    }
}
```

`quickSort({4, 8, 2, 9, 3, 7, 1})` · Zustand je zerlege

Tiefe	zerlege	p	Reihung danach	Vgl.
0	(0, 6)	3	1 2 3 4 8 7 9	6
1	(0, 2)	0	1 2 3 4 8 7 9	2
2	(1, 2)	1	1 2 3 4 8 7 9	1
1	(4, 6)	5	1 2 3 4 7 8 9	2
Summe				**11**

Vergleiche bei n = 1000 (gemessen)

Verfahren	sortiert	gemischt	Speicher
Selectionsort	499 500	499 500	in-place
Mergesort	4 932	8 716	neue Reihungen
Quicksort	499 500	10 325	in-place

Quicksort mit Pivot links · gemischt: Pseudozufall wie in Welt 2

Schlechtester Fall: sortierte Eingabe – das Pivot ist immer das Minimum: n(n − 1)/2 Vergleiche (n = 6: 15).

Faustregel: Halbieren ist der Trick – Mergesort teilt immer gleichmäßig.

Unbekannte rekursive Verfahren analysieren

1. **Kopf und Basisfall lesen**
   Wann endet die Rekursion? Was wird übergeben?

2. **Teilproblem bestimmen**
   Wie ändern sich die Parameter beim Selbstaufruf?

3. **Kleines Beispiel tracen**
   Rekursionstabelle mit Tiefe und Belegung

4. **Aufrufe zählen**
   für n = 1, 2, 3, 4 – Muster erkennen, Formel aufstellen

5. **Deuten und beurteilen**
   Was leistet das Verfahren? Kennst du es schon?

Beispiel: die geheime Methode

```
public static void geheim(int[] r, int i) {
    if (i < r.length - 1) {
        int m = i;
        for (int j = i + 1; j < r.length; j++) {
            if (r[j] < r[m]) {
                m = j;
            }
        }
        int h = r[i];
        r[i] = r[m];
        r[m] = h;
        geheim(r, i + 1);
    }
}
```

r = {5, 8, 2, 6}

Tiefe	Aufruf	m	r danach
0	geheim(r, 0)	2	2 8 5 6
1	geheim(r, 1)	2	2 5 8 6
2	geheim(r, 2)	3	2 5 6 8
3	geheim(r, 3)	-	Basisfall

Abi-Format: Rekursionsverlauf dokumentieren

Aufgabe [3 BE]

Gegeben ist r = {3, 8, 2, 5}. Dokumentieren Sie alle Aufrufe von summeAb, die beim Aufruf summeAb(r, 0) erfolgen, in der Reihenfolge ihres Beginns. Geben Sie zu jedem Aufruf die Rekursionstiefe und den Rückgabewert an.

```
public static int summeAb(int[] r, int i) {
    if (i == r.length) {
        return 0;                    // Basisfall
    }
    return r[i] + summeAb(r, i + 1);
}
```

Musterantwort

Rekursionstiefe	Aufruf	i	r[i]	Rückgabe
0	summeAb(r, 0)	0	3	3 + 15 = 18
1	summeAb(r, 1)	1	8	8 + 7 = 15
2	summeAb(r, 2)	2	2	2 + 5 = 7
3	summeAb(r, 3)	3	5	5 + 0 = 5
4	summeAb(r, 4)	4	–	0 (Basisfall)

- Tiefe ab 0 – der Erstaufruf zählt mit
- auch den Basisfall-Aufruf summeAb(r, 4) notieren
- Rückgaben von unten nach oben ausrechnen
- Ergebnis nennen: summeAb(r, 0) liefert 18

Boss 3: Die Kristallhöhle



DS 23

60 Minuten · 24 BE
Zeitbudget wie im Abi: ≈ 2,5 min pro BE

Papier, ohne Gerät
erlaubt sind nur deine Werkzeugkarten

Neuer Kontext
tief unter dem Labyrinth: Rekursion verfolgen, analysieren, entwerfen

Danach
Papier-Compiler in der IDE · zusätzlich im Lab: Online-Boss „Das Echo der Schlucht“

Vorbereitung: Warm-up-Karten W3-01 bis W3-14 · Werkzeugkarten · Rekursionstiefe: Erstaufruf = 0